Undo implementation specification

Use **multi-level undo with complete address-book snapshots**, following the Developer Guide's proposed design. Policies marked **recommended** fill gaps in the guide.

Source: docs/DeveloperGuide.md, line 213: [Proposed] Undo/redo feature.

1. Command behavior

- Syntax: `undo`, with no arguments. Allow surrounding whitespace; reject arguments such as `undo 2`.
- Undo the most recent successful command that changed stored application data.
- Each invocation reverses exactly one command. Repeated invocations step backwards through earlier changes.
- Success message: `Undo successful.`
- If no earlier state exists, throw `CommandException` with `No commands to undo.` Data, history, filters, and sorting must remain unchanged.
- Undo itself must not create a new history entry.

2. Commands covered

Category	Commands
Student records	`add`, `edit`, `delete`, `clear`
Class enrolment	`enroll`, `unenroll`
Attendance	`attendance`, `attendance unmark`
Assignments and marks	`assignment`, `mark`, both forms of `removeassignment`

`find`, `filter`, `list`, `sort`, `view`, `marks`, `help`, and `exit` do not create history entries.

A command that changes several related records must be undone as one operation. For example, undoing a student deletion must restore the student **and their assignment memberships and marks**.

3. Snapshot contents

Each snapshot must contain:

- Every student's name, matriculation number, email, tags, classes, and attendance.
- All assignment definitions, student-assignment links, and marks.
- The distinction between an unrecorded mark and a recorded zero.

Use independent copies of mutable data. Later changes must never alter earlier snapshots.

The existing AddressBook copy/reset implementation (line 62) already copies the student collection and reconstructs the gradebook. Immutable `Person` objects may be shared between snapshots; mutable gradebook collections must be copied.

4. History and model API

Create `VersionedAddressBook extends AddressBook`, containing:

- `List<AddressBook> addressBookStateList`
- `int currentStatePointer`

Initialize history with one independent snapshot of the loaded application data and pointer `0`.

Operation	Contract
`commit()`	Discard states after the current pointer, append a snapshot of current data, then point to it.
`canUndo()`	Return whether `currentStatePointer > 0`.
`undo()`	Restore the snapshot immediately before the current state and move the pointer back once. Reject calls when undo is unavailable.

Expose these through `Model` as `commitAddressBook()`, `canUndoAddressBook()`, and `undoAddressBook()`.

`ModelManager` must own the versioned address book. Restore data into the existing observable collection so the UI's list bindings continue working.

This specification covers undo; the list-and-pointer design also supports adding redo later.

5. Recording changes

Each modifying command must call `commitAddressBook()` exactly once after all its data changes succeed, before returning its result.

Do not commit inside individual setters: one command may invoke several model operations.

- Parsing and validation failures must leave data and history unchanged.
- A command must either complete all its data changes or leave the previous data intact.
- **Recommended:** Successful commands that leave data unchanged create no entry. Examples include clearing already-empty data or setting an existing mark to the same value.
- Detect unchanged data by comparing complete address-book contents, including the gradebook.
- Check for unchanged data **before** discarding any later history.
- After undo followed by a new data change, the abandoned later states must be discarded.

6. Session, display, and saving policies - recommended

- Keep history in memory for the current application session. On restart, the loaded data becomes the new initial state.
- Initially retain all snapshots for the session. Document that memory usage grows with the number and size of changes.
- After successful undo, show all students using the default name ordering and clear assignment-sort details. This prevents filters hiding restored students or assignment sorting displaying stale marks.
- Close an open student-details window after restoration so it cannot display outdated data.
- Save the restored state through the normal storage flow. Persist current application data only, without history.

The current LogicManager saves after executing commands (line 62). If saving fails, retain the changed in-memory data and matching history position, and report that the change was applied in memory but could not be saved. This storage failure policy is distinct from a parsing or validation failure.

7. Required acceptance tests

Scenario	Expected result
Undo immediately after startup	Error; nothing changes.
Two changes followed by two undos	Restore each preceding state, then the initial state.
Another undo at the initial state	Error; pointer remains valid.
Change -> read-only command or failed command -> undo	Reverse the successful data change.
Change -> successful no-op -> undo	Reverse the actual change.
Undo -> new change -> undo	Return to the state immediately before the new change.
Delete student -> undo	Restore all student fields, assignment links, and marks.
Edit matriculation number -> undo	Restore the original identity and gradebook keys.
Create-and-assign assignment -> undo	Reverse creation and assignment together.
Assign existing assignment -> undo	Remove the new link while preserving the shared definition.
Record/update mark -> undo	Restore the previous mark or unmarked state.
Either form of removeassignment, or clear -> undo	Restore all affected records without altering unrelated data.
Modify data after taking a snapshot	Earlier snapshots remain unchanged.
Undo with filtering or assignment sorting active	Correct restored data, all students visible, default sorting.
Save failure during undo	Clear error; in-memory data and history remain consistent.
Restart after a saved undo	Load restored data with no previous-session undo history.

Also test valid whitespace, rejected arguments, parser routing, and observable-list updates.

8. Completion criteria

Add `UndoCommand`, register it in `AddressBookParser`, integrate history into the model and modifying commands, and add model, command, parser, and storage-flow tests.

Update the Developer Guide's proposed status once implemented, document undo in the User Guide, and remove its existing statement that no undo command exists. Run the relevant tests and style checks.

The history-boundary tests apply the NUS CS2103/T AY2026/27 Semester 1 textbook's equivalence partitioning and boundary value analysis. The specific application policies above are project recommendations.